



◆ OFFENSIVE SECURITY SERIES ◆

ATTACK SURFACE MAPPING

The Complete Enumeration Playbook

Nmap · Masscan · Nuclei · Amass · HTTPX · Aquatone · Shodan · Cloud ASM

Nmap

Masscan

Nuclei

Amass

HTTPX

Aquatone

Nikto

Feroxbuster

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

Attack Surface Mapping — The Complete Enumeration Playbook

01.	ATTACK SURFACE MAPPING FUNDAMENTALS
02.	NETWORK SCANNING — NMAP DEEP DIVE
03.	MASSCAN — INTERNET-SCALE PORT DISCOVERY
04.	SUBDOMAIN ENUMERATION — AMASS, SUBFINDER & DNSX
05.	WEB APPLICATION FINGERPRINTING — HTTPX & WHATWEB
06.	VISUAL RECON — AQUATONE, EYEWITNESS & GOWITNESS
07.	VULNERABILITY SCANNING — NUCLEI & NIKTO
08.	WEB CONTENT DISCOVERY — GOBUSTER, FFUF & FERROXBUSTER
09.	TLS/SSL & CERTIFICATE ANALYSIS
10.	CLOUD ATTACK SURFACE — AWS, AZURE & GCP
11.	API ATTACK SURFACE DISCOVERY
12.	GOOGLE DORK — ADVANCED OPERATOR REFERENCE
13.	GITHUB DORKING & SOURCE CODE RECON
14.	ASSET INVENTORY & CONTINUOUS ASM PLATFORMS
15.	FULL ASM WORKFLOW CHEAT SHEET



01 — ATTACK SURFACE MAPPING FUNDAMENTALS

Scope, Methodology & The Four ASM Dimensions

Attack Surface Mapping (ASM) is the systematic process of discovering, cataloguing, and analysing every possible entry point an attacker could use against a target. It covers network assets, web applications, cloud infrastructure, APIs, third-party integrations, and human exposure. A complete ASM phase transforms a raw scope into a prioritised list of targets with known services, versions, and potential vulnerabilities.

Dimension	What It Covers	Primary Tools
Network Layer	IPs, ports, protocols, firewalls, VPN endpoints, routers	Nmap, Masscan, Shodan
Web Layer	Domains, subdomains, web apps, APIs, login pages, JS files	Amass, HTTPX, Aquatone
Application Layer	CMS, frameworks, software versions, CVEs, misconfigs	Nuclei, Nikto, WhatWeb
Cloud Layer	S3 buckets, storage blobs, open GCS, serverless functions	CloudBrute, Bucket Finder
Human Layer	Employee emails, credentials in code, leaked secrets	GitHub Dorking, Trufflehog

ASM METHODOLOGY — ORDERED APPROACH

1. Passive Discovery — WHOIS, crt.sh, Shodan (no target contact)
2. Active DNS — Subdomain brute-force, zone transfer attempts
3. Port & Service Scan — TCP/UDP full-range with version detection
4. Web Fingerprinting — HTTP probing, tech stack, headers, screenshots
5. Vuln Correlation — Match discovered versions to known CVEs
6. Cloud Enumeration — S3/Blob/GCS, serverless, IAM misconfigs
7. API Discovery — Swagger docs, JS endpoint extraction, fuzzing
8. Reporting — Prioritised findings by exploitability & impact



02 — NETWORK SCANNING — NMAP DEEP DIVE

Port Discovery, Service Detection, NSE Scripts & Output Formats

Nmap is the backbone of network-layer ASM. Beyond basic port scanning, Nmap's NSE (Nmap Scripting Engine) provides 600+ scripts for service enumeration, vulnerability detection, credential brute-force, and protocol-specific discovery. Mastering Nmap scan types, timing options, and output formats is essential for every red teamer.

Core Nmap Scan Types

Flag	Scan Type	Use Case	Noise Level
-sS	TCP SYN (Stealth)	Default fast scan, half-open connections	Low
-sT	TCP Connect	Full 3-way handshake — logged by target	High
-sU	UDP Scan	DNS, SNMP, TFTP, NTP discovery	Low
-sA	TCP ACK	Firewall rule mapping, stateful detection	Medium
-sN/-sF/-sX	NULL/FIN/Xmas	IDS evasion on some systems	Low
-sV	Version Detection	Service/version banner grabbing	Medium
-sC	Default NSE Scripts	Combines safe discovery scripts	Medium
-O	OS Detection	TTL + TCP stack fingerprinting	Medium
-A	Aggressive	-sV -sC -O --traceroute combined	High

\$ Nmap — Practical Red Team Commands

```
# Phase 1: Quick discovery — top 1000 ports, all hosts in range
nmap -sS -T4 --open -oA nmap_quick 192.168.1.0/24

# Phase 2: Full TCP port scan on discovered live hosts
nmap -sS -p- -T4 --open -oA nmap_fullTCP -iL live_hosts.txt

# Phase 3: Service + version + default scripts on open ports
nmap -sV -sC -p 22,80,443,8080,8443,3389,5985 -oA nmap_svc 10.10.10.1

# Phase 4: UDP top ports (slow but critical for SNMP/DNS/NTP)
nmap -sU --top-ports 200 -T4 -oA nmap_udp 10.10.10.0/24

# Phase 5: OS detection + aggressive on single host
nmap -A -T4 -p- -oA nmap_full 10.10.10.1

# Firewall evasion techniques
nmap -f -D RND:10 --source-port 53 -T2 -sS 10.10.10.1
nmap --data-length 25 -T2 --randomize-hosts -sS 10.10.10.0/24

# Parse Nmap XML for open ports
python3 -c "import xml.etree.ElementTree as ET; .."
grep 'open' nmap_quick.gnmap | awk '{print $2}' > live_hosts.txt
```

Key NSE Script Categories

Category	Script Examples	Target Service
auth	ftp-anon, http-auth-finder, smb-enum-users	FTP, HTTP, SMB
vuln	http-vuln-cve*, smb-vuln-ms17-010, ssl-heartbleed	Web, SMB, TLS
discovery	dns-zone-transfer, snmp-info, http-headers	DNS, SNMP, HTTP
brute	ftp-brute, http-brute, smb-brute, ssh-brute	All credential services
exploit	http-shellshock, ftp-proftpd-backdoor	Known CVEs

\$ NSE Script Usage Examples

```
# Run all vuln scripts against web server
nmap --script 'vuln' -p 80,443 10.10.10.1

# SMB enumeration — shares, users, OS
nmap --script smb-enum-shares,smb-enum-users,smb-os-discovery -p 445 10.10.10.1

# Check EternalBlue (MS17-010)
nmap --script smb-vuln-ms17-010 -p 445 10.10.10.1

# SNMP enumeration
nmap -sU -p 161 --script snmp-info,snmp-sysdescr,snmp.interfaces 10.10.10.1

# HTTP full enumeration
nmap --script http-headers,http-methods,http-title,http-robots.txt -p 80,443 10.10.10.1

# DNS zone transfer attempt
nmap --script dns-zone-transfer --script-args dns-zone-transfer.domain=target.com

# SSL/TLS cipher enumeration
nmap --script ssl-enum-ciphers -p 443 10.10.10.1
```



03 — MASSCAN — INTERNET-SCALE PORT DISCOVERY

Async Packet Engine, Rate Tuning & Large Subnet Scanning

Masscan is an asynchronous TCP port scanner capable of scanning the entire internet in under 6 minutes. It uses its own TCP/IP stack (bypassing the OS) and sends packets at millions per second. For red team engagements involving large IP ranges or external perimeter scans, Masscan provides rapid discovery that Nmap can then target for deep service enumeration.

\$ Masscan — Core Usage

```
# Install masscan
sudo apt install masscan

# Basic scan — top ports on /24
sudo masscan 192.168.1.0/24 -p 22,80,443,8080,3389 --rate=10000

# Full TCP port scan on single host
sudo masscan 10.10.10.1 -p 0-65535 --rate=100000 -oJ masscan_full.json

# Large range — common ports fast
sudo masscan 10.0.0.0/8 -p 22,80,443,445,3389,8080,8443 --rate=50000 \
  --output-format grepable --output-filename masscan.gnmap

# Exclude ranges (important for scope compliance)
sudo masscan 10.0.0.0/8 -p 80,443 --excludefile exclude.txt --rate=50000

# Output formats: -oJ (JSON), -oX (XML), -oG (grepable), -oL (list)

# Parse masscan JSON output — extract IP:port pairs
python3 -c "
import json
data = json.load(open('masscan_full.json'))
for h in data: print(f\"{h['ip']}:{h['ports'][0]['port']}\")
"

# Feed masscan output to nmap for service detection
nmap -sV -sC -iL <(grep open masscan.gnmap | awk '{print $2}') -oA nmap_svc
```

MASSCAN RATE GUIDELINES

- Rate 10,000 — Safe for most network environments, low IDS trigger risk
- Rate 100,000 — Fast internal scans, may trigger rate-based IDS
- Rate 1,000,000 — Internet-scale only, will cause packet loss on most LANs
- Tip: --rate=500 for ultra-stealth scans through monitored perimeters
- Tip: Use --adapter-ip and --router-mac if scanning through specific interface



04 — SUBDOMAIN ENUMERATION

Amass, Subfinder, DNSX & Permutation Scanning

Subdomain enumeration expands the web attack surface dramatically. Forgotten subdomains host legacy applications, unpatched services, and staging environments with weaker security. The modern approach chains passive certificate discovery with active brute-force and DNS resolution to build a comprehensive verified subdomain list.

\$ Subdomain Enumeration Pipeline

```
# 1. Passive — certificate transparency (no DNS queries to target)
curl -s 'https://crt.sh/?q=%target.com&output=json' | \
jq -r '.[].name_value' | sort -u > crtsh_subs.txt

# 2. Amass — comprehensive passive + active enumeration
amass enum -passive -d target.com -o amass_passive.txt
amass enum -active -d target.com -brute -w /usr/share/wordlists/seclists/
Discovery/DNS/subdomains-top1million-5000.txt -o amass_active.txt

# 3. Subfinder — fastest passive subdomain tool
subfinder -d target.com -silent -all -o subfinder.txt

# 4. Merge and deduplicate all results
cat crtsh_subs.txt amass_passive.txt amass_active.txt subfinder.txt | \
sort -u > all_subs.txt

# 5. DNS resolution — only keep live subdomains
dnsx -l all_subs.txt -silent -a -resp -o live_subs.txt

# 6. Permutation scanning — find variations (dev, staging, uat, etc.)
gotator -sub live_subs.txt -perm permutations.txt -depth 1 -silent | \
dnsx -silent -a -resp -o permutation_subs.txt

# 7. Final live count
wc -l live_subs.txt
[*] Total live subdomains: 847
```

Tool	Type	Best For	Speed
Amass	Passive + Active	Most comprehensive — 50+ data sources	Slow
Subfinder	Passive	Fast passive — API key powered	Fast
dnsx	DNS resolver	Verify live subdomains at scale	Very Fast
MassDNS	DNS brute-force	Millions of DNS lookups per minute	Extreme
Gotator	Permutations	dev, staging, uat, api, test variants	Fast
crt.sh	Cert Transparency	Best for passive cert-based discovery	Fast



Tool	Type	Best For	Speed
Puredns	DNS brute-force	Accurate at high concurrency	Very Fast



05 — WEB APPLICATION FINGERPRINTING

HTTPX, WhatWeb, Wappalyzer & Response Analysis

Web fingerprinting identifies running technologies, versions, frameworks, and server configurations from HTTP responses. This intelligence directly maps to exploitable CVEs, default credentials, and known misconfigurations. Every discovered subdomain should be probed for HTTP/HTTPS responses before deeper testing begins.

\$ HTTPX — Mass HTTP Probing

```
# Probe all discovered subdomains for HTTP/HTTPS
httpx -l live_subs.txt -silent -status-code -title -tech-detect \
  -content-length -web-server -o httpx_results.txt

# Full-featured probe with screenshots (requires chromium)
httpx -l live_subs.txt -screenshot -o httpx_full -mc 200,301,302,401,403

# Find specific technologies
httpx -l live_subs.txt -tech-detect | grep -i 'wordpress\|drupal\|joomla'
httpx -l live_subs.txt -tech-detect | grep -i 'apache\|nginx\|iis'

# Extract page titles — find login pages, admin panels
httpx -l live_subs.txt -title | grep -i 'login\|admin\|portal\|dashboard'

# Find open redirects via header analysis
httpx -l live_subs.txt -follow-redirects -location -silent

# WhatWeb — CMS and framework detection
whatweb -i live_subs.txt --log-json whatweb_results.json -a 3
whatweb https://target.com -v # verbose single target

# Grab all HTTP response headers
curl -sI https://target.com | grep -i 'server\|x-powered-by\|x-aspnet'

[*] https://admin.target.com [200] [Admin Panel] [Apache/2.4.51] [PHP/7.4]
[*] https://staging.target.com [200] [Staging Env] [Nginx/1.18] [WordPress/5.8]
[*] https://api.target.com [200] [API Gateway] [Express] [Node.js]
```

Header / Indicator	What It Reveals	Attack Implication
Server: Apache/2.4.49	Exact Apache version	Check CVE-2021-41773 Path Traversal
X-Powered-By: PHP/7.2	PHP version	Check known PHP CVEs, deserialization
X-Generator: WordPress 5.8	CMS + version	wpscan, known plugin CVEs
Set-Cookie: PHPSESSID	PHP session cookie	Session fixation, PHP app testing
Access-Control-Allow-Origin: *	Wildcard CORS	CORS exploitation for data theft



Header / Indicator	What It Reveals	Attack Implication
X-Frame-Options: missing	No clickjacking protection	Clickjacking attack possible
Strict-Transport-Security: missing	No HSTS	SSL stripping possible



06 — VISUAL RECON

Aquatone, EyeWitness & GoWitness — Screenshot All The Things

Visual recon automates screenshot capture of every live web asset. When dealing with hundreds of subdomains, manually browsing each is impractical. These tools capture screenshots of every HTTP/HTTPS service and generate an HTML report that lets you instantly identify login portals, admin panels, exposed dev environments, and misconfigured applications.

\$ Visual Recon Tools

```
# GoWitness – fastest screenshot tool
go install github.com/sensepost/gowitness@latest

# Screenshot all subdomains from file
gowitness file -f live_subs.txt --delay 2 --screenshot-path ./screenshots
gowitness report generate # HTML report at report.html

# Nmap XML input (screenshot all discovered web services)
gowitness nmap -f nmap_full.xml --open --service-contains http

# Aquatone – pipe httpx/nmap output to screenshots
cat live_subs.txt | aquatone -out aquatone_report -screenshot-timeout 3000
# Output: aquatone_report/aquatone_report.html

# EyeWitness – Python-based, supports RDP/VNC too
python3 EyeWitness.py -f live_subs.txt --web --timeout 10 -d eyewitness_out
python3 EyeWitness.py --xml nmap_full.xml --web -d eyewitness_out

# HTTPX with screenshot (built-in, no extra tool needed)
httpx -l live_subs.txt -screenshot -system-chrome -o httpx_screenshots

# What to look for in screenshots:
# - Login portals → test default creds
# - Admin panels → /admin, /wp-admin, /manager
# - Dev/staging → weaker security, real data
# - Error pages → version info, path disclosure
# - Citrix/OWA → high-value remote access
```

HIGH-VALUE VISUAL TARGETS

Outlook Web Access (OWA) — Password spray + 2FA bypass attempts

Citrix NetScaler / ADC — CVE history, Gateway exploitation

VPN Login Pages — Pulse, Fortinet, GlobalProtect, Cisco AnyConnect

Jenkins / GitLab / Jira — DevOps toolchain, code access, secret leaks

phpMyAdmin / Adminer — Direct DB access if default creds work

Exposed Grafana / Kibana — Internal metrics, log data, possible RCE

Staging / Dev environments— Often same codebase, weaker auth or config



07 — VULNERABILITY SCANNING — NUCLEI & NIKTO

Template-Driven CVE Detection & Web Server Auditing

Nuclei is the modern standard for automated vulnerability detection. It uses YAML templates — community-contributed or custom — to detect CVEs, misconfigurations, exposed panels, default credentials, and sensitive data exposure across thousands of technologies. With 9,000+ public templates and a growing community, Nuclei dramatically accelerates the transition from 'asset discovered' to 'exploitable finding'.

\$ Nuclei — Automated Vuln Scanning

```
# Install and update templates
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
nuclei -update-templates

# Run all templates against discovered web assets
nuclei -l httpx_results.txt -o nuclei_results.txt -severity critical,high,medium

# Focused scan — CVEs only
nuclei -l httpx_results.txt -tags cve -severity critical,high -o cve_findings.txt

# Default credentials check
nuclei -l httpx_results.txt -tags default-login -o default_creds.txt

# Exposed panels and misconfigurations
nuclei -l httpx_results.txt -tags panel,exposure,misconfiguration

# Specific CVE check
nuclei -u https://target.com -id CVE-2021-44228 # Log4Shell
nuclei -u https://target.com -id CVE-2022-22965 # Spring4Shell
nuclei -u https://target.com -id CVE-2021-26084 # Confluence RCE

# Rate limiting for stealth
nuclei -l targets.txt -rate-limit 10 -concurrency 5 -timeout 10

# Custom template usage
nuclei -l targets.txt -t /path/to/custom/templates/ -o custom_findings.txt

# Nikto — traditional web server scanner
nikto -h https://target.com -o nikto_results.txt -Format htm
nikto -h https://target.com -Tuning 9 -ssl # Focus on SQL injection
```

Nuclei Tag	What It Checks	Priority
cve	Known CVEs matched to detected tech versions	Critical
default-login	Default username/password combos	Critical

Nuclei Tag	What It Checks	Priority
exposure	Exposed files, config dumps, API keys, tokens	High
panel	Admin panels, login portals, management UIs	High
misconfiguration	CORS, headers, open redirects, debug mode	Medium
takeover	Subdomain takeover vulnerable CNAMEs	High
tech	Technology detection and version fingerprinting	Info



08 — WEB CONTENT DISCOVERY

Gobuster, FFUF, Feroxbuster & Wordlist Strategy

Content discovery finds hidden directories, files, endpoints, and functionality not linked from the main application. Backup files, admin panels, API docs, configuration files, and forgotten upload paths are all regularly found through wordlist-based fuzzing. The right wordlist for the target technology stack dramatically increases findings.

\$ Content Discovery Commands

```
# Gobuster — directory and file brute force
gobuster dir -u https://target.com -w /usr/share/seclists/Discovery/
  Web-Content/raft-large-directories.txt -x php,html,txt,bak,zip \
  -t 50 -o gobuster_dirs.txt

# Gobuster — DNS subdomain mode
gobuster dns -d target.com -w /usr/share/seclists/Discovery/
  DNS/subdomains-top1million-5000.txt -t 50 -o gobuster_dns.txt

# FFUF — fastest fuzzer, supports all injection points
ffuf -u https://target.com/FUZZ -w raft-large-directories.txt \
  -mc 200,301,302,403 -t 100 -o ffuf_dirs.json -of json

# FFUF — fuzz parameters
ffuf -u 'https://target.com/page?FUZZ=test' \
  -w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt

# FFUF — vhost discovery
ffuf -u https://target.com -H 'Host: FUZZ.target.com' \
  -w subdomains-top1million-5000.txt -mc 200 -fs 0

# Feroxbuster — recursive content discovery
feroxbuster -u https://target.com -w raft-medium-directories.txt \
  -x php,asp,aspx,html,txt -r -t 50 --depth 3 -o ferox_results.txt

# Key SecLists wordlists to use:
# raft-large-directories.txt — broad directory coverage
# common.txt — most common files/dirs
# api/api-endpoints.txt — REST API endpoint names
# burp-parameter-names.txt — GET/POST parameter names
```

HIGH-VALUE PATHS TO CHECK

`/.git/config, /.git/HEAD` — Exposed git repo → source code dump

`/.env, /config.php, /wp-config.php` — Environment files with DB creds

`/backup.zip, /backup.tar.gz` — Full application backup files

`/api/v1/, /api/swagger.json` — API documentation and endpoints

`/admin/, /administrator/, /manage` — Admin panels

`/phpinfo.php, /info.php` — PHP configuration disclosure

`/.htpasswd, /web.config` — Server config / credential files



09 — TLS/SSL & CERTIFICATE ANALYSIS

sslyze, testssl.sh & Certificate Intelligence

TLS/SSL analysis identifies weak ciphers, deprecated protocol versions, certificate misconfigurations, and vulnerabilities like BEAST, POODLE, ROBOT, and Heartbleed. Certificates also expose internal hostnames, related domains, and organisational structure through Subject Alternative Names (SANs).

\$ TLS/SSL Analysis Commands

```
# testssl.sh – comprehensive TLS testing
testssl.sh --full https://target.com
testssl.sh --vulnerable https://target.com # Check all CVEs only
testssl.sh --severity HIGH https://target.com # Only high+ findings

# sslyze – Python-based TLS scanner
pip install sslyze
python3 -m sslyze --regular target.com:443
python3 -m sslyze --heartbleed target.com:443

# OpenSSL – manual certificate inspection
openssl s_client -connect target.com:443 </dev/null 2>/dev/null | \
  openssl x509 -noout -text | grep -E 'Subject:|DNS:|Not After'

# Extract all SANs from certificate (reveals other hostnames)
openssl s_client -connect target.com:443 2>/dev/null | \
  openssl x509 -noout -ext subjectAltName

# Check certificate chain
openssl s_client -connect target.com:443 -showcerts 2>/dev/null

# Nmap SSL enumeration
nmap --script ssl-enum-ciphers,ssl-heartbleed,ssl-poodle -p 443 target.com

# Find all certs for org via crt.sh API
curl -s 'https://crt.sh/?q=target.com&output=json' | \
  jq -r '.[ ] | .name_value' | sort -u
```



10 — CLOUD ATTACK SURFACE

AWS, Azure, GCP — Bucket Discovery & IAM Misconfigs

Cloud infrastructure introduces unique attack surface: publicly exposed storage buckets, misconfigured serverless functions, exposed metadata services, and overly permissive IAM roles. S3 buckets named after the company domain are a common high-value find — containing backup files, source code, and internal docs.

\$ Cloud Attack Surface Discovery

```
# S3 bucket enumeration — common naming patterns
# Patterns: target, target-backup, target-prod, target-dev, target-assets

# AWS — check if bucket exists and is public
aws s3 ls s3://target --no-sign-request
aws s3 ls s3://target-backup --no-sign-request
aws s3 sync s3://target-backup . --no-sign-request # Download if public!

# CloudBrute — automated bucket brute force across AWS/Azure/GCP
git clone https://github.com/0xsha/CloudBrute && go build
cloudbrute -d target.com -k target -m wordlist.txt -t 80 -T 10 -c providers.yaml

# Bucket Finder — S3 bucket discovery
bucket_finder --download --region us-east-1 target_wordlist.txt

# GrayhatWarfare — passive public bucket search
# https://buckets.grayhatwarfare.com/

# Azure Blob Storage enumeration
az storage blob list --account-name target --container-name public \
  --auth-mode anonymous --output table
# Try: target.blob.core.windows.net
curl https://target.blob.core.windows.net/?comp=list # List containers

# GCP Storage bucket check
gsutil ls gs://target
curl https://storage.googleapis.com/target/?list # Public check

# AWS metadata endpoint (if you have SSRF on EC2)
curl http://169.254.169.254/latest/meta-data/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

Cloud Provider	Key Attack Surface	Discovery Method
AWS	S3 buckets, EC2 metadata, Lambda URLs, API GW	aws CLI, CloudBrute, GrayhatWarfare
Azure	Blob storage, App Services, Function URLs, AD	az CLI, MicroBurst, AADInternals

Cloud Provider	Key Attack Surface	Discovery Method
GCP	GCS buckets, Cloud Functions, Firebase, BigQuery	gsutil, GCPBucketBrute
All	Exposed env vars, hardcoded keys in JS/git	trufflehog, gitleaks, nuclei



11 — API ATTACK SURFACE DISCOVERY

Swagger, JS Endpoint Extraction & API Fuzzing

APIs represent one of the fastest-growing attack surfaces in modern applications. REST, GraphQL, and SOAP endpoints often lack the same security controls as web UIs. API discovery involves finding documentation endpoints, extracting endpoints from JavaScript bundles, and identifying API gateways through HTTP analysis.

\$ API Discovery Techniques

```
# Common API documentation paths to check
# /api/swagger.json, /swagger/v1/swagger.json
# /api-docs, /openapi.json, /v1/api-docs
# /graphql, /graphiql, /api/graphql

# Gobuster — API endpoint discovery
gobuster dir -u https://target.com/api -w api-endpoints.txt \
  -x json,xml -t 50 -o api_endpoints.txt

# FFUF — REST API version fuzzing
ffuf -u https://target.com/api/FUZZ -w api-versions.txt -mc 200,401,403

# Extract API endpoints from JavaScript files
# Step 1: Find all JS files
gau target.com | grep '\.js$' | sort -u > js_files.txt

# Step 2: Extract endpoints from JS with LinkFinder
python3 linkfinder.py -i https://target.com -d -o linkfinder_out.html

# Step 3: Use JSluice for structured JS analysis
cat js_files.txt | xargs -I{} curl -s {} | jsluice urls

# GraphQL introspection (if enabled = security misconfiguration)
curl -X POST https://target.com/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name fields{name}}}}"}' | jq

# Kiterunner — API route brute force with real API wordlists
kr scan https://target.com -w routes-small.kite -o kr_results.txt
```



12 — GOOGLE DORKING

Advanced Operator Reference for Attack Surface Discovery

Dork	Finds
site:target.com filetype:pdf	PDFs — often contain internal data
site:target.com filetype:xlsx OR filetype:csv	Spreadsheets — employee data, financials
site:target.com inurl:admin OR inurl:login	Admin and login pages
site:target.com inurl:wp-admin	WordPress admin panel
site:target.com "Index of /"	Open directory listings
site:target.com ext:env OR ext:config OR ext:bak	Config/backup files
site:target.com "phpinfo()" OR "PHP Version"	PHP info pages
site:target.com "sql syntax" OR "mysql_fetch"	SQL error disclosure
intext:"@target.com" site:pastebin.com	Pastes with target emails
"target.com" "password" site:pastebin.com	Leaked credentials in pastes
site:target.com -www	Non-www subdomains indexed by Google
inurl:"target.com/api" filetype:json	Exposed API JSON endpoints
site:target.com intitle:"Dashboard" OR intitle:"Kibana"	Internal dashboards
"target.com" "secret_key" OR "api_key" site:github.com	Secrets in GitHub



13 — GITHUB DORKING & SOURCE CODE RECON

Secrets, Keys, Internal Paths & Leaked Credentials

\$ GitHub Dorking Queries

```
# GitHub search operators (use at github.com/search)

# Find API keys / secrets
org:TargetCorp password
org:TargetCorp secret_key
org:TargetCorp api_key language:python
org:TargetCorp AWS_SECRET_ACCESS_KEY
org:TargetCorp 'BEGIN RSA PRIVATE KEY'

# Find internal hostnames and IPs
org:TargetCorp 'internal.target.com'
org:TargetCorp '10.0.0' OR '192.168'

# Config and connection strings
org:TargetCorp connectionString
org:TargetCorp jdbc:mysql
org:TargetCorp smtp_password

# Automated scanning with Trufflehog
trufflehog github --org=TargetCorp --json > github_secrets.json
trufflehog github --repo=https://github.com/TargetCorp/repo --json

# Gitleaks – scan cloned repos
git clone https://github.com/TargetCorp/app.git
gitleaks detect --source ./app --report-format json -r leaks.json

# Gitrob – multi-repo secret scanner (legacy but useful)
gitrob analyse TargetCorp
```



14 — ASM PLATFORMS & CONTINUOUS MONITORING

Asset Inventory, Change Tracking & Enterprise ASM

Modern red team engagements and bug bounty hunting benefit from continuous ASM — automated asset discovery that runs on a schedule and alerts on new assets, new open ports, and new CVE matches. Several open-source and commercial platforms provide this capability at scale.

Platform	Type	Key Features	Cost
ProjectDiscovery Cloud	SaaS/OS	Nuclei, HTTPX, Subfinder integration	Free tier
Spiderfoot HX	Self-hosted	200+ modules, scheduled scans	Paid
OWASP Amass	Open Source	Most comprehensive passive/active enum	Free
Shodan Monitor	SaaS	Continuous port monitoring on your IPs	Paid
Censys Attack Surface	SaaS	Certificate + port monitoring	Paid
Chaos (PD)	Open Source	Bug bounty program subdomain dataset	Free
reconFTW	Open Source	Full recon automation pipeline script	Free

\$ reconFTW — Full Automated Pipeline

```
# reconFTW chains all tools automatically
git clone https://github.com/six2dez/reconftw.git
cd reconftw && ./install.sh

# Full reconnaissance on domain
python3 reconftw.py -d target.com -r -a -o ./results/

# What reconFTW runs automatically:
# amass + subfinder + dnsx → subdomain enum + resolution
# httpx → HTTP probing
# gowitness → screenshots
# nuclei → vuln scanning
# gau + waybackurls → historical URLs
# ffuf / feroxbuster → content discovery
# trufflehog → secret scanning
# nmap → port scanning on live hosts
# And 20+ more tools...

# Output: /results/target.com/ with categorised findings
[*] Subdomains found: 847
[*] Live hosts: 312
[*] Nuclei findings: 43 (12 High, 4 Critical)
[*] Secrets found: 7
```



15 — FULL ASM WORKFLOW CHEAT SHEET

Complete Command Reference — From Scope to Findings

WHOIS + ASN	<code>whois target.com && curl api.bgpview.io/search?query_term=TargetCorp</code>
Cert transparency	<code>curl 'crt.sh/?q=%target.com&output=json' jq -r '.[].name_value'</code>
Passive subdomains	<code>subfinder -d target.com -silent -all -o subs.txt</code>
Active subdomain enum	<code>amass enum -active -d target.com -brute -w subdomains.txt</code>
DNS resolution	<code>dnsx -l subs.txt -silent -a -resp -o live_subs.txt</code>
Permutation subs	<code>gotator -sub live_subs.txt -perm permutations.txt dnsx</code>
Fast port scan	<code>sudo masscan -p 0-65535 10.10.10.0/24 --rate=100000 -oJ scan.json</code>
Nmap service scan	<code>nmap -sV -sC -p- -T4 -oA nmap_full -iL live_hosts.txt</code>
HTTP probing	<code>httpx -l live_subs.txt -status-code -title -tech-detect -o httpx.txt</code>
Screenshots	<code>gowitness file -f live_subs.txt --screenshot-path ./screenshots</code>
Nuclei CVE scan	<code>nuclei -l httpx.txt -tags cve -severity critical,high -o nuclei.txt</code>
Default creds check	<code>nuclei -l httpx.txt -tags default-login -o defaultcreds.txt</code>
Dir bruteforce	<code>feroxbuster -u https://target.com -w raft-large.txt -x php,txt -r</code>
API endpoint fuzz	<code>ffuf -u https://target.com/api/FUZZ -w api-endpoints.txt -mc 200</code>
JS endpoint extract	<code>python3 linkfinder.py -i https://target.com -d -o links.html</code>
GraphQL introspect	<code>curl -X POST https://target.com/graphql -d '{"query":"{__schema{types{name}}}"}</code>
S3 bucket check	<code>aws s3 ls s3://target-backup --no-sign-request</code>
Cloud brute	<code>cloudbrute -d target.com -k target -m wordlist.txt -t 80</code>
GitHub secrets	<code>trufflehog github --org=TargetCorp --json > secrets.json</code>
SSL/TLS audit	<code>testssl.sh --vulnerable https://target.com</code>
Full auto pipeline	<code>python3 reconftw.py -d target.com -r -a -o ./results/</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.